Data Mining

Week 3

Submitter - Himanshu Singh

Sentiment Analysis and Preprocessing Text

Download the labeled training dataset from this link: Bag of Words Meets Bags of Popcorn.

Load the dataset as a Pandas data frame.

In [5]:
```
import os
import pandas as pd

# Set the path to the file you'd like to load
file_path = "data/movie_data"
```

Part 1: Using the TextBlob Sentiment Analyzer

- Import the movie review data as a data frame and ensure that the data is loaded properly.
- How many of each positive and negative reviews are there?
- Use TextBlob to classify each movie review as positive or negative. Assume that a polarity score greater than or equal to zero is a positive sentiment and less than 0 is a negative sentiment.
- Check the accuracy of this model. Is this model better than random guessing?
- For up to five points extra credit, use another prebuilt text sentiment analyzer, e.g., VADER, and repeat steps (3) and (4).

In [6]:
```
import pandas as pd
from textblob import TextBlob
from nltk.sentiment.vader import SentimentIntensityAnalyzer
```

```python
from sklearn.metrics import accuracy_score
import nltk

# Download VADER lexicon
nltk.download('vader_lexicon', quiet=True)

# Import the movie review data as a data frame and ensure that the data is loaded properly.

# loading the file in dataframe
df = pd.read_csv(os.path.join(file_path, "labeledTrainData.tsv"), sep="\t"  )

# Verify the data is loaded properly
print(df.head())
print(df.info())

# How many of each positive and negative reviews are there?

# The column is named 'sentiment'
review_counts = df['sentiment'].value_counts()
#print("Actual Review Counts:\n", review_counts)

pos_count = review_counts[1]
neg_count = review_counts[0]

print(f"Number of Positive Reviews: {pos_count}")
print(f"Number of Negative Reviews: {neg_count}")

# Use TextBlob to classify each movie review as positive or negative.
# Assume that a polarity score greater than or equal to zero is a positive sentiment
# and less than 0 is a negative sentiment.

# Function to get TextBlob sentiment
def get_textblob_sentiment(text):
    analysis = TextBlob(str(text))
    # Return 1 for positive, 0 for negative
    return 1 if analysis.sentiment.polarity >= 0 else 0

# Apply to the dataframe
df['textblob_pred'] = df['review'].apply(get_textblob_sentiment)

tb_accuracy = accuracy_score(df['sentiment'], df['textblob_pred'])
```

```python
print(f"TextBlob Model Accuracy: {tb_accuracy:.2%}")

# Check the accuracy of this model. Is this model better than random guessing?

# Random Guessing Baseline: In a balanced binary dataset (50% positive, 50% negative),
# random guessing yields an accuracy of 50%.
# Comparison
baseline = 0.50
if tb_accuracy > baseline:
    print("This model is better than random guessing.")
else:
    print("This model performs similarly to or worse than random guessing.")

# Now, use the VADER sentiment analyzer to classify each movie review as positive or negative.
# Again, assume that a compound score greater than or equal to zero is a positive sentiment
# and less than 0 is a negative sentiment.

analyzer = SentimentIntensityAnalyzer()

def get_vader_sentiment(text):
    analyzer = SentimentIntensityAnalyzer()
    score = analyzer.polarity_scores(str(text))
    return 1 if score['compound'] >= 0 else 0

df['vader_pred'] = df['review'].apply(get_vader_sentiment)

vader_accuracy = accuracy_score(df['sentiment'], df['vader_pred'])
print(f"VADER Model Accuracy: {vader_accuracy:.2%}")
```

```
       id  sentiment                                             review
0  5814_8          1  With all this stuff going down at the moment w...
1  2381_9          1  \The Classic War of the Worlds\" by Timothy Hi...
2  7759_3          0  The film starts with a manager (Nicholas Bell)...
3  3630_4          0  It must be assumed that those who praised this...
4  9495_8          1  Superbly trashy and wondrously unpretentious 8...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   id         25000 non-null  object
 1   sentiment  25000 non-null  int64
 2   review     25000 non-null  object
dtypes: int64(1), object(2)
memory usage: 586.1+ KB
None
Number of Positive Reviews: 12500
Number of Negative Reviews: 12500
TextBlob Model Accuracy: 68.52%
This model is better than random guessing.
VADER Model Accuracy: 69.36%
```

Part 2: Prepping Text for a Custom Model

- If you want to run your own model to classify text, it needs to be in proper form to do so. The following steps will outline a procedure to do this on the movie reviews text.
- Convert all text to lowercase letters.
- Remove punctuation and special characters from the text.
- Remove stop words.
- Apply NLTK's PorterStemmer.
- Create a bag-of-words matrix from your stemmed text (output from (4)), where each row is a word-count vector for a single movie review (see sections 5.3 & 6.8 in the Machine Learning with Python Cookbook). Display the dimensions of your bag-of-words matrix. The number of rows in this matrix should be the same as the number of rows in your original data frame.

- Create a term frequency-inverse document frequency (tf-idf) matrix from your stemmed text, for your movie reviews (see section 6.9 in the Machine Learning with Python Cookbook). Display the dimensions of your tf-idf matrix. These dimensions should be the same as your bag-of-words matrix.

In [7]:
```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

# Download necessary NLTK data
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
ps = PorterStemmer()

# Function to clean text

def clean_text(text):
    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. & 4. Tokenize, remove stop words, and apply PorterStemmer
    tokens = text.split()
    cleaned_tokens = [ps.stem(word) for word in tokens if word not in stop_words]

    return " ".join(cleaned_tokens)

# Apply the cleaning process
df['cleaned_review'] = df['review'].apply(clean_text)

print(df[['review', 'cleaned_review']].head())

# Create Bag-of-Words matrix from stemmed text

# Initialize CountVectorizer
```

```python
count_vectorizer = CountVectorizer()

# Fit and transform the cleaned text
bow_matrix = count_vectorizer.fit_transform(df['cleaned_review'])

# Display dimensions
print(f"Bag-of-Words Matrix Dimensions: {bow_matrix.shape}")

# Create TF-IDF matrix from stemmed text

# Initialize TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit and transform the cleaned text
tfidf_matrix = tfidf_vectorizer.fit_transform(df['cleaned_review'])

# Display dimensions
print(f"TF-IDF Matrix Dimensions: {tfidf_matrix.shape}")
```

```
                                              review  \
0  With all this stuff going down at the moment w...
1  \The Classic War of the Worlds\" by Timothy Hi...
2  The film starts with a manager (Nicholas Bell)...
3  It must be assumed that those who praised this...
4  Superbly trashy and wondrously unpretentious 8...

                                       cleaned_review
0  stuff go moment mj ive start listen music watc...
1  classic war world timothi hine entertain film ...
2  film start manag nichola bell give welcom inve...
3  must assum prais film greatest film opera ever...
4  superbl trashi wondrous unpretenti exploit hoo...
Bag-of-Words Matrix Dimensions: (25000, 89468)
TF-IDF Matrix Dimensions: (25000, 89468)
```